



STUDENT

0014-MYJ

TENTAMEN

250324 DV2606 2110

Salstentamen (DO)

Kurskod	--
Bedömningsform	--
Starttid	24.03.2025 14:00
Sluttid	24.03.2025 19:00
Bedömningsfrist	--
PDF skapad	05.11.2025 17:14
Skapad av	Anders Göransson

1.1 *SPMD* is a common programming model for writing parallel programs. *SPMD* stands for:

- ☐ Shared-memory Program Many Distributions
- ☐ Single Processor Multiple threads
- ☐ Shared-memory Programming for Multiple Data
- ☒ Single Program Multiple Data
- ☐ None of the alternatives is correct.



Rätt. 1 av 1 poäng.

1.2 The cpu execution time can be described as $T_{exe} = I * CPI * T_c$. Which of the the following alternatives lower the CPI, thus *increasing* the performance?

- ☐ Increase the cache miss latency.
- ☐ Use fewer more complex instructions instead of more simple instructions.
- ☐ Decrease the cache hit ratio.
- ☐ None of the alternatives is correct.

☒ Increase the instruction-level parallelism.



Rätt. 1 av 1 poäng.

1.3 Assume a parallel application that is running on a parallel system with a certain efficiency (processor utilization). Which of the following alternatives would *increase* the efficiency?

- ☒ Increase the input data set size.
- ☐ Decrease the input data set size.
- ☐ Increase the number of processors.
- ☐ Put more barrier synchronizations in the code.
- ☐ None of the alternatives increases the efficiency.



Rätt. 1 av 1 poäng.

1.4 Assume a parallel application that is running on a parallel system with a certain efficiency (processor utilization). Which of the following alternatives would *decrease* the efficiency?

- ☐ Put more barrier synchronizations in the code.
- ☐ Increase the number of processors.
- ☒ All alternatives decrease the efficiency.
- ☐ Decrease the input data set size.
- ☐ Increase the clock frequency of the processors.



Rätt. 1 av 1 poäng.

1.5 What is the (asymptotic) parallel time complexity of *matrix-vector multiplication* using n processors, assuming an $n \times n$ matrix and an $n \times 1$ vector?

- ☐ $O(n^2)$
- ☐ $O(\log n)$
- ☐ None of the alternatives is correct.
- ☒ $O(n)$
- ☐ $O(n \log n)$



Rätt. 1 av 1 poäng.

1.6 Simultaneous multithreading is ...

- ☒ a technique to exploit parallelism in a distributed memory computer
- ☐ none of the alternatives is correct
- ☐ a technique to improve branch prediction
- ☐ a technique to implement multi-level caches
- ☐ a software approach to parallelize an application



Fel. 0 av 1 poäng.

1.7 Why are snoopy-based cache coherence protocols not used in large-scale shared-memory multiprocessors?

- ☒ They are not scalable to a large number of nodes. 
- ☐ They can only be implemented using a write-update policy.
- ☐ None of the alternatives is correct.
- ☐ They can not guarantee coherence for more than ten caches.
- ☐ Modern processors are too fast for snoopy-based protocols.

Rätt. 1 av 1 poäng.

1.8 MSI (Modified-Shared-Invalid) is a basic write-invalidate protocol. Assuming that a particular cache block is in state Modified, which action is performed when the cache sees a read request on the shared bus?

- ☐ None of the alternatives is correct. 
- ☐ The cache replies with the requested block, and the block stays in state Modified.
- ☐ The cache replies with the requested block, and the block is marked as Shared. 
- ☐ The request is denied since the block is in state Modified.
- ☐ The processor is always stalled until the bus request is handled.

Fel. 0 av 1 poäng.

1.9 In a MIMD parallel computer, the different processors (or processor nodes) are connected through an *interconnection network*. Which of the following metric(s) is(are) used when evaluating the performance of an interconnection network?

☐ Bisection bandwidth

☒ Diameter



☐ All of the mentioned metrics are valid evaluation metrics



☐ Bisection width

☐ Connectivity

Fel. 0 av 1 poäng.

1.10 How many neighbors does each internal node have in a 2-dimensional mesh network with $n \times n$ nodes, i.e., in total n^2 nodes?

☐ n

☒ n^2



☐ 4



☐ $n \log n$

☐ $\log n$

Fel. 0 av 1 poäng.

- 2.1** Explain why we need to *protect shared data* in shared memory systems.
 Give an example what could happen if we do not protect shared data.
 Discuss some common ways to protect shared data in shared memory systems.

shared memory is when processors share the same memory and threads communicate through memory location in the shared memory bus. we need to protect the data so not other thread overwrite this data

If we don't protect it other thread may overwrite the data in the memory that the previous thread handled.

we can also use privatization technique so each thread gets private copies of the data, in this way we can protect the shared data in the shared memory system

we can use also atomic operation so read modify write can be handled by one thread and no other thread disturb, in this way we can protect the shared data in the shared memory system

Ord: 116

Besvarad.

- 2.2** Memory consistency is an important concept for a shared-memory multiprocessor. Clearly describe what memory consistency means and why it is important. (2p)
 Describe two different memory consistency models, including clearly describe how they differ, their advantages and disadvantages, etc. (2p)

memory consistency define the rules of memory operation(read/write) that executed from different multiple threads in the shared memory, it ensure all threads has consistent view of memory and its important because its crucial for correctness, performance and portability.

two models are:

sequential consistency: memory operation executes in a single global order

advantage: easy to reason about, it don't need synchronization

disadvantage: limited the performance due strict order

Relaxed consistency: memory operation may reordered or delayed, except at synchronization point

advantage: high performance and scalability due ordering

disadvantage: hard to reason about, need synchronization

Ord: 93

Besvarad.

2.3 You are working at BTH Embedded System Technologies (BEST), a startup company from BTH. At BEST, you are responsible for developing a new embedded system based on multicore processors and parallel programming. Since the company develops both the hardware and the software, you can decide how many cores that are needed for a particular task. The company management expects the new parallel system to reach a performance speedup of at least 10 times for the program given below as compared to the previous system that was based on a uniprocessor. You are *not allowed to change the program*.

Assume that:

- Each core can complete one floating point operation per clock cycle.
- Cores are clocked at 2 GHz, and each have a 1 MB cache using FIFO replacement.
- The time to service a cache miss (i.e., the time to load the block into the cache) is 200ns.
- All BEST processors (both singlecore and multicore) are attached to a 200 GB/s memory bus.
- Memory latency is perfectly hidden (excellent prefetchers!).

===== start of code =====

```
#define SIZE 1024
```

```
static double A[SIZE][SIZE], B[SIZE][SIZE], C[SIZE][SIZE];
```

```
int main(int argc, char** argv)
```

```
{
```

```
    int i, j, k;
```

```
    double sum, average;
```

```
    // Assume that A and B are initialized before the timer starts (not shown)
```

```
    // Semantics of "parallel_for":
```

```
    // Loop iterations are perfectly partitioned using blocked-cyclic assignment
```

```
    // among P threads each running on one of the processor's P cores.
```

```
    // Timer starts here
```

```
    parallel_for (i = 0; i < SIZE; i++) {
```

```
        for (j = 0; j < SIZE; j++) {
```

```
            c[i][j] = 0.0;
```

```
            for (k = 0; k < SIZE; k++)
```

```
                c[i][j] = c[i][j] + a[i][k] * b[k][j];
```

```
        }
```

```
    }
```

```
    sum = 0.0;
```

```
    for (i = 0; i < SIZE; i++) {
```

```
        for (j = 0; j < SIZE; j++)
```

```
            sum = sum + c[i][j];
```

```
        average = sum / (SIZE * SIZE);
```

```
    // Timer stops here
```

```
}
```

===== end of code =====

a. (3p)

How do you respond to this design challenge? Do you believe you can meet the performance goal?

- If yes, how many cores should be included in the new parallel system?
- If no, explain why.

(justify your answer using e.g. calculations / numbers)

b. (4p)

If you were allowed you to make a few changes to the code, you may be able to deliver better speedup with your multicore processor design. How would you change the code to improve its performance (i.e., improve its speedup)? (describe by using e.g., pseudo code)

- If your answer was NO in part one, how many processors are required to achieve 10 times speedup now?
- If your answer was YES, approximately what speedup do you expect from your previously proposed machine on the new code?

(justify your answer using e.g. calculations / numbers)

Ord: 0

Obesvarad.

3.1

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

☐ Accesses to GPU global memory are faster than accesses to GPU shared memory.

☒ Threads in different thread blocks cannot access the same variable in GPU shared memory 

☐ All the threads in a thread block see the same registers

☒ The content of the CUDA global memory is not changed between two kernel launches in the same program 

☐ The CPU can access variables in GPU shared memory

Rätt. 2 av 2 poäng.

3.2

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- ☐ The CPU can load values into the constant memory on the GPU ✓
- ☐ Constant memory cannot contain floating point numbers
- ☒ Threads in different blocks cannot see the same variables in constant memory ✗
- ☐ Variables in constant memory are automatically moved to registers
- ☐ Variables in constant memory can be cached ✓

Fel. 0 av 2 poäng.

3.3 Thread Blocks and Warps

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- ☒ Threads in the same Warp cannot execute different instructions at the same time. ✓
- ☐ Threads in the same Warp can execute on different Streaming Multiprocessors.
- ☐ A Thread Block cannot be larger than a Warp.
- ☒ A Thread Block can contain many Warps. ✓
- ☐ Threads from different Thread Blocks can execute in the same Warp.

Rätt. 2 av 2 poäng.

3.4 Scatter to Gather

Scatter to gather conversion is a technique that can be used for improving the performance of some CUDA program. Explain how scatter to gather conversion works and why this technique can improve performance. Also give an example of scatter to gather conversion.

scatter: write operation to arbitrary memory location.

gather: reads operation from arbitrary memory location.

conversion: transform from scatter pattern(inefficient) to gather pattern (efficient) by reorganization of data access.

how can improve performance:

scatter: cause non coalesced memory writes leading to poor memory bandwidth utilization.

gather: ensure coalesced memory reads

example:

```
__global__ void scatter(int output, int input, int indice)
```

```
    int idx: threadIdx.x
```

```
    output[indice[idx]] = input[idx]  (non-coalesced)
```

```
__global__ void gather(int output, int input, int indice)
```

```
    int idx: threadIdx.x
```

```
    output[idx] = input[indice[idx]] (coalesced)
```

Ord: 78

Besvarad.

3.5

Occupancy is defined as the ratio of active warps on an SM to the maximum number of active warps supported by the SM. Why do we normally want to have high occupancy? Which hardware related parameters affect occupancy? Discuss and give examples of how we as programmers, by taking smart design decisions, can provide good trade-offs between the parameters that affect occupancy.

high occupancy is desirable because it hides latency and improves resource utilization

occupancy is affected by hardware related parameters such as register usage, memory shared usage, block size, GPU architecture limits

programmers parameters that affect occupancy by reduce the register and memory shared usage, choose appropriate block size and profiling their codes.

Ord: 52

Besvarad.

3.6

What is control divergence? Why is control divergence bad for performance? Give an example how we as programmers can avoid or minimize control divergence.

Ord: 0

Obesvarad.

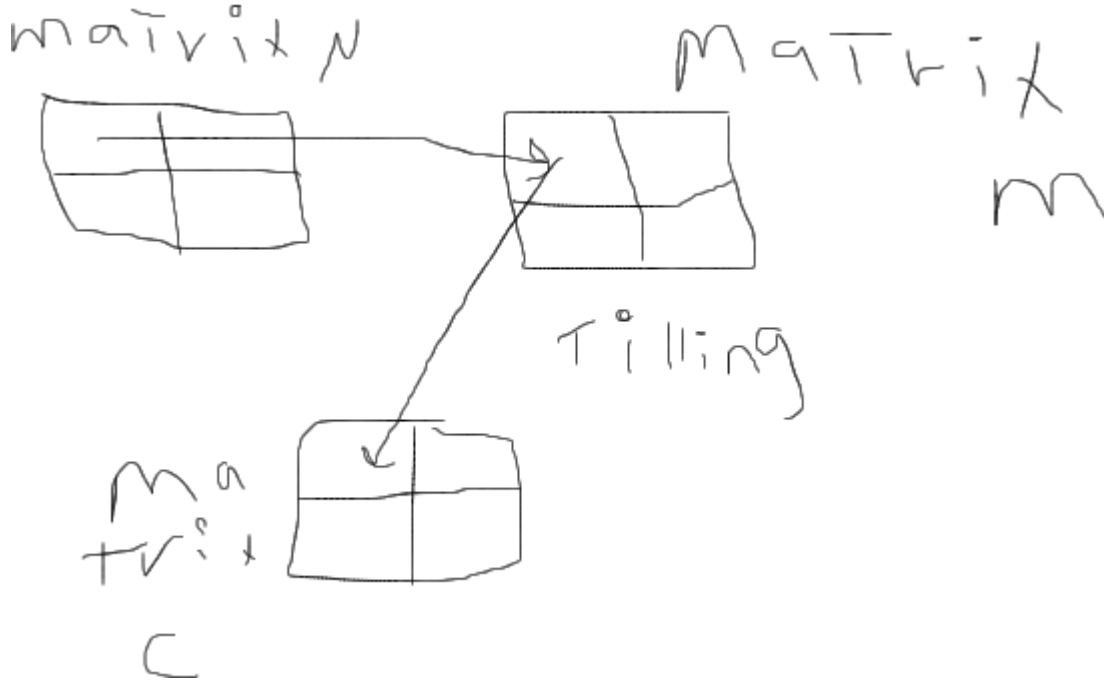
3.7 Tiling

Tiling is a technique that can be used for improving the performance of some CUDA programs, e.g., programs that perform parallel matrix multiplication or convolution. Explain how tiling works, why it can increase performance and why and how we may need to use `__syncthreads()` when using tiling.

tiling is dividing the data into smaller tiles, threads load these tiles computing result locally and write it back to the global memory.

tiling reduce global memory access, use shared memory because its faster and more coalesced . tiling ensure efficient global memory use shared memory efficiency so no conflicts occur. we use `__syncthreads()` so all the thread finishes loading tiles before the program computing begin and that no race condition occur.

Example: matrix multiplication tiles of data loaded into shared memory and we use `__syncthreads()` for the same same purpose mentioned before



Ord: 93

Besvarad.

3.8

Explain how sparse matrices are stored when using the ELL format. Also explain the potential performance advantages for a CUDA program when using the ELL format compared to the CSR (compressed sparse row) format. Also, explain how sparse matrices are stored when using the COO (coordinate list) format. Also, explain why we sometimes would like to combine the ELL and COO formats when using CUDA. Give an example of how the ELL and COO formats can be combined when storing a sparse matrix.

COO enables regularization by padding by addition of dummy elements(e.g, zeros) to the sparse matrix representation. this is done by appending padding entries to values, row indice and column indice array. for example in 3x3 sparse matrix this padding ensure that each row has at least 2 entries

the flexibility of COO makes it suitable for application requires consistent matrix structure for example memory alignment and parallel processing

Ord: 68

Besvarad.

3.9 Thread coarsening

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- ☐ Thread coarsening makes it possible to perform data transfer between the CPU and GPU in parallel with kernel execution on the GPU
- ☐ Thread coarsening increases the parallelism in a CUDA program
- ☒ Thread coarsening can reduce the number redundant calculations in a CUDA program ✓
- ☐ Thread coarsening decreases the completion time of a thread
- ☒ Thread coarsening can make it possible to use the shared memory more efficiently in a CUDA program ✓

Rätt. 2 av 2 poäng.